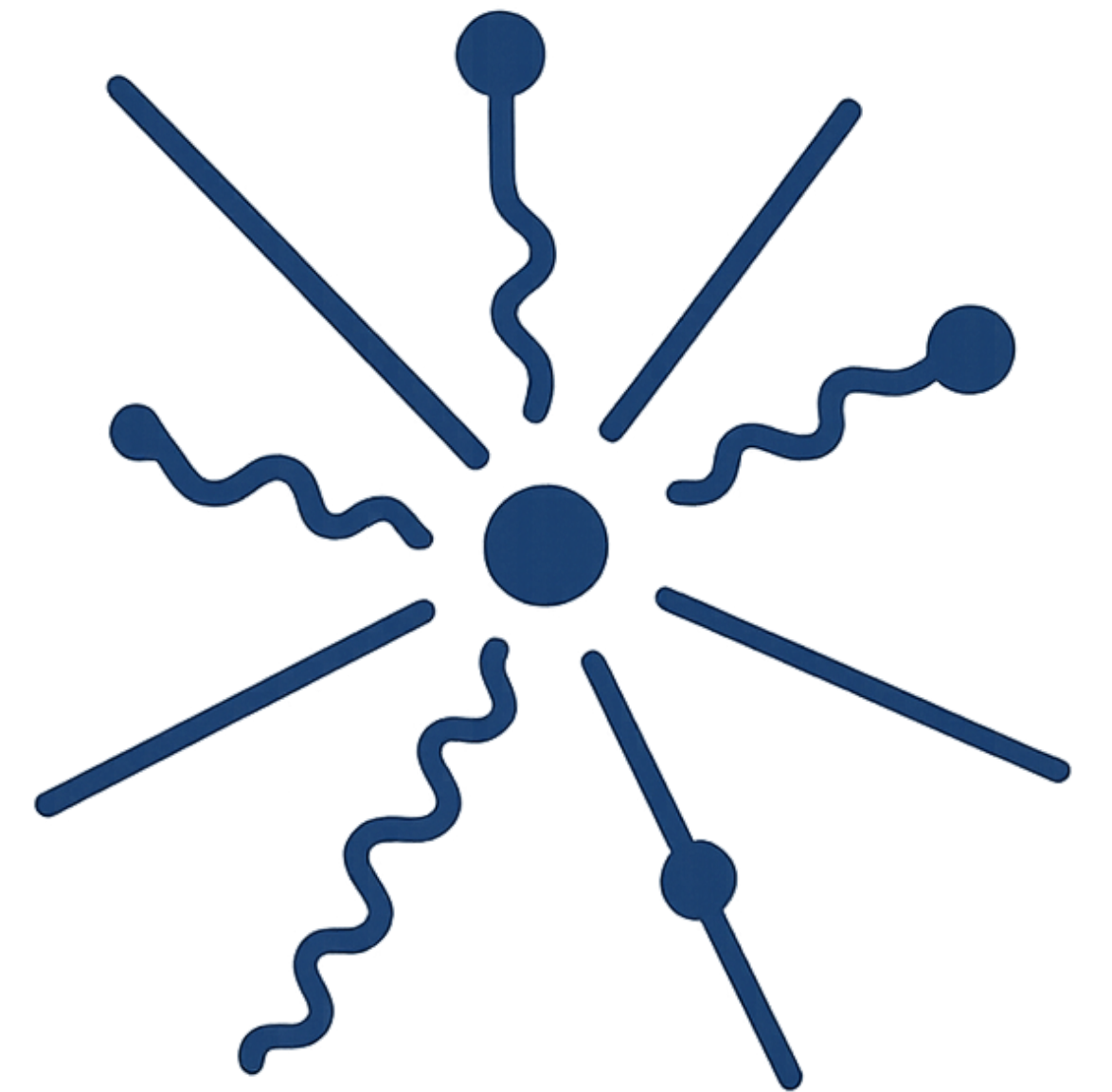
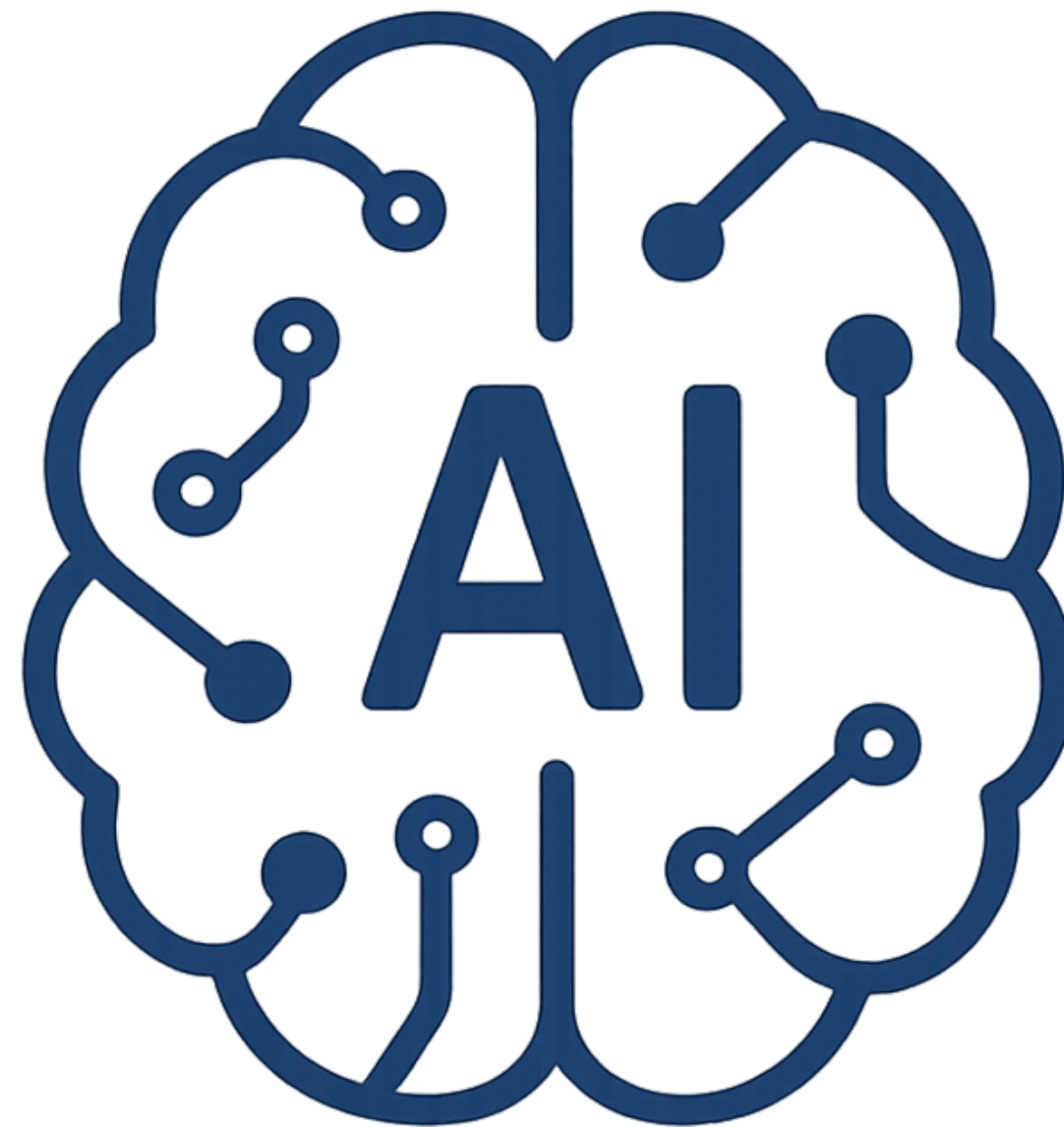
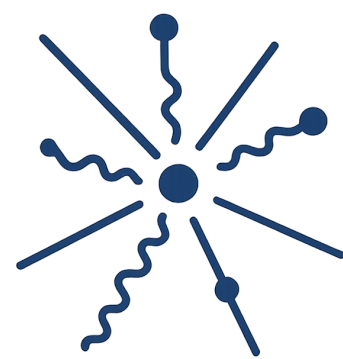
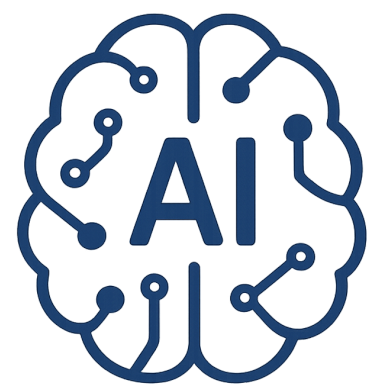


Introduction to

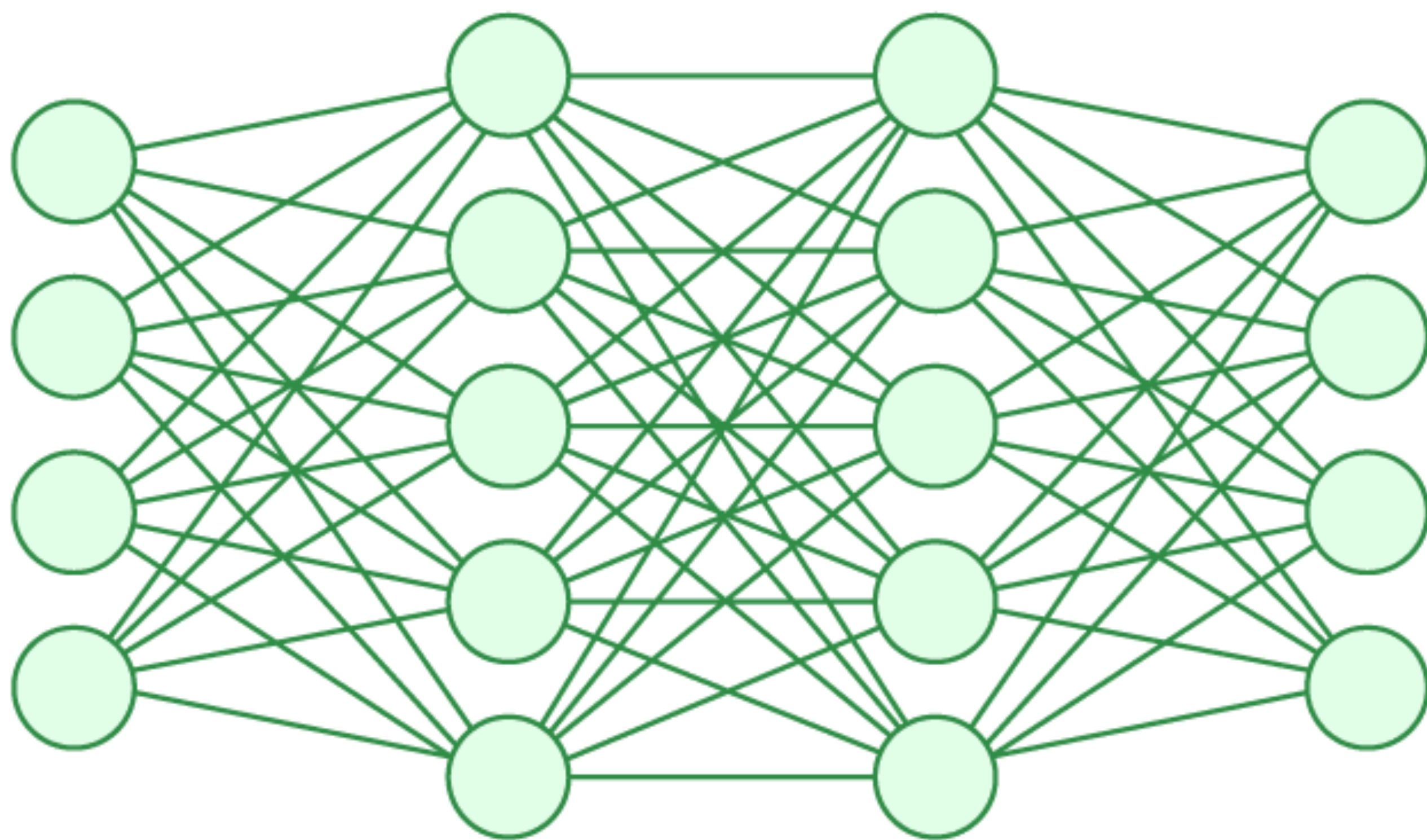
AI-Driven HEP

S. A. Fard - School of Physics (IPM)





AI-Driven HEP 10



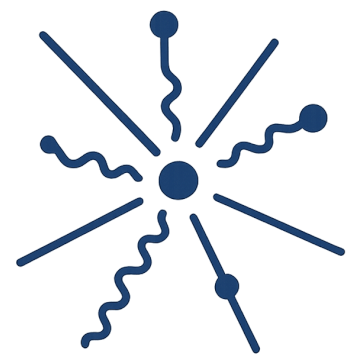
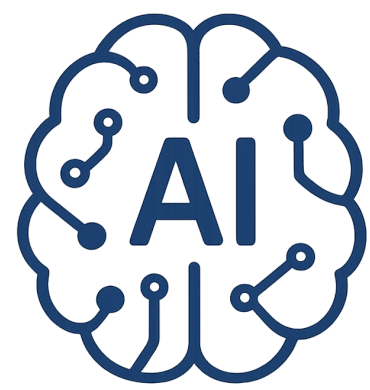
Session 10

Neural Networks II

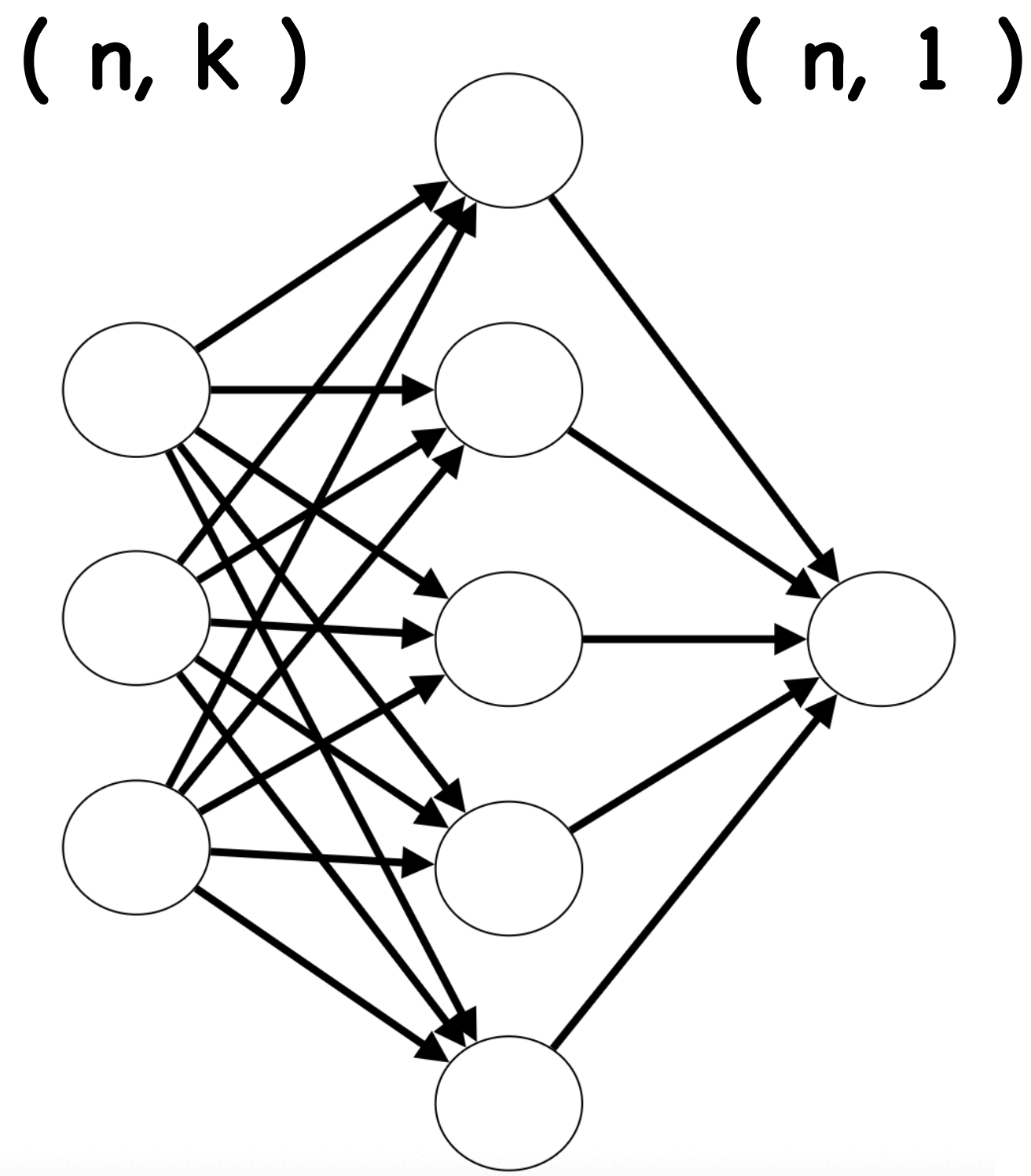


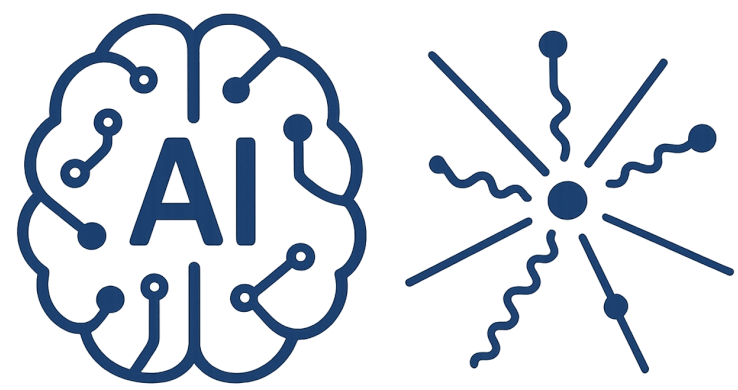
Reference

- [1] Bishop, Christopher M., and Hugh Bishop. Deep learning: Foundations and concepts. Springer Nature, 2023.
- [2] <https://www.ruder.io/optimizing-gradient-descent/>

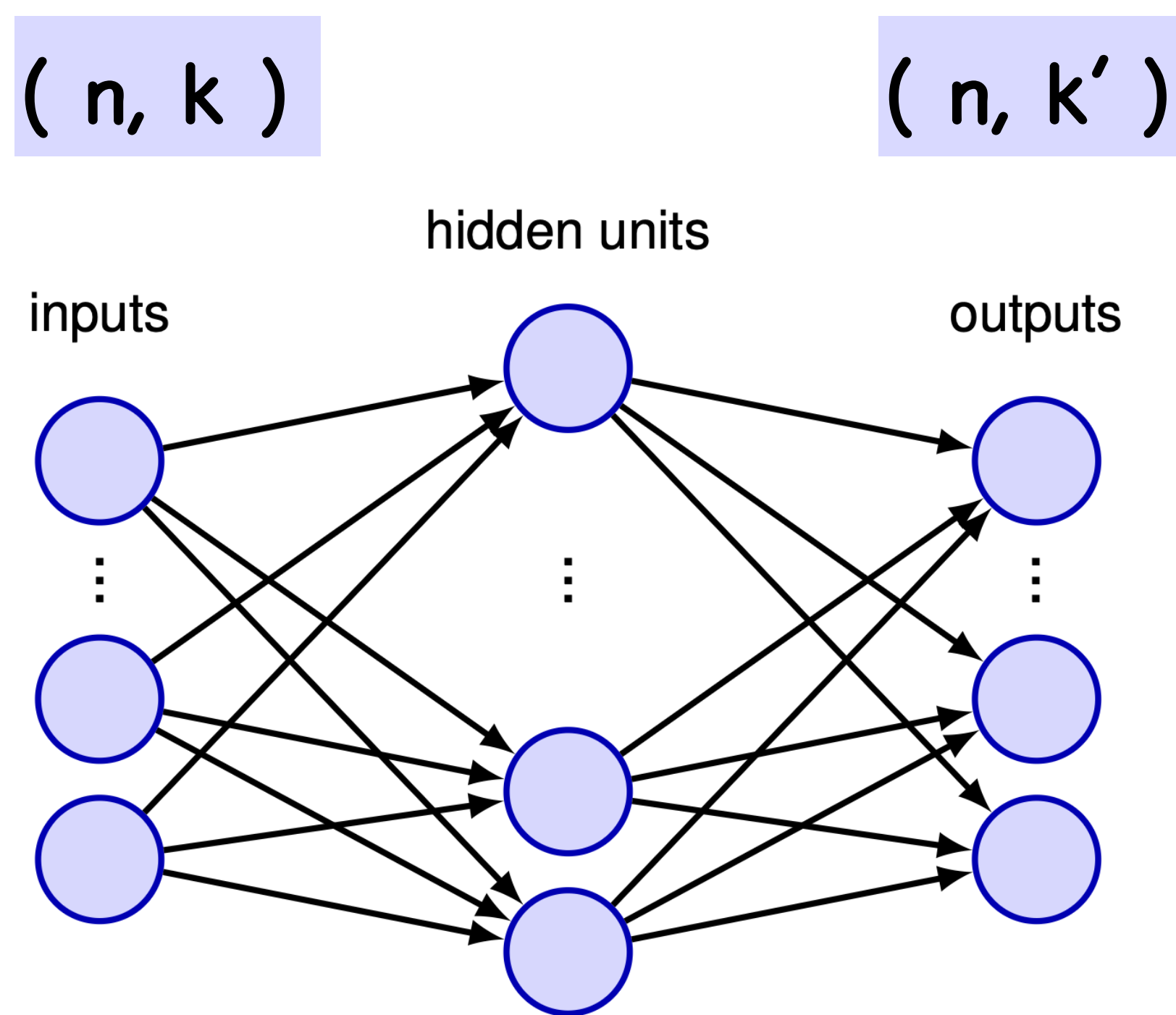
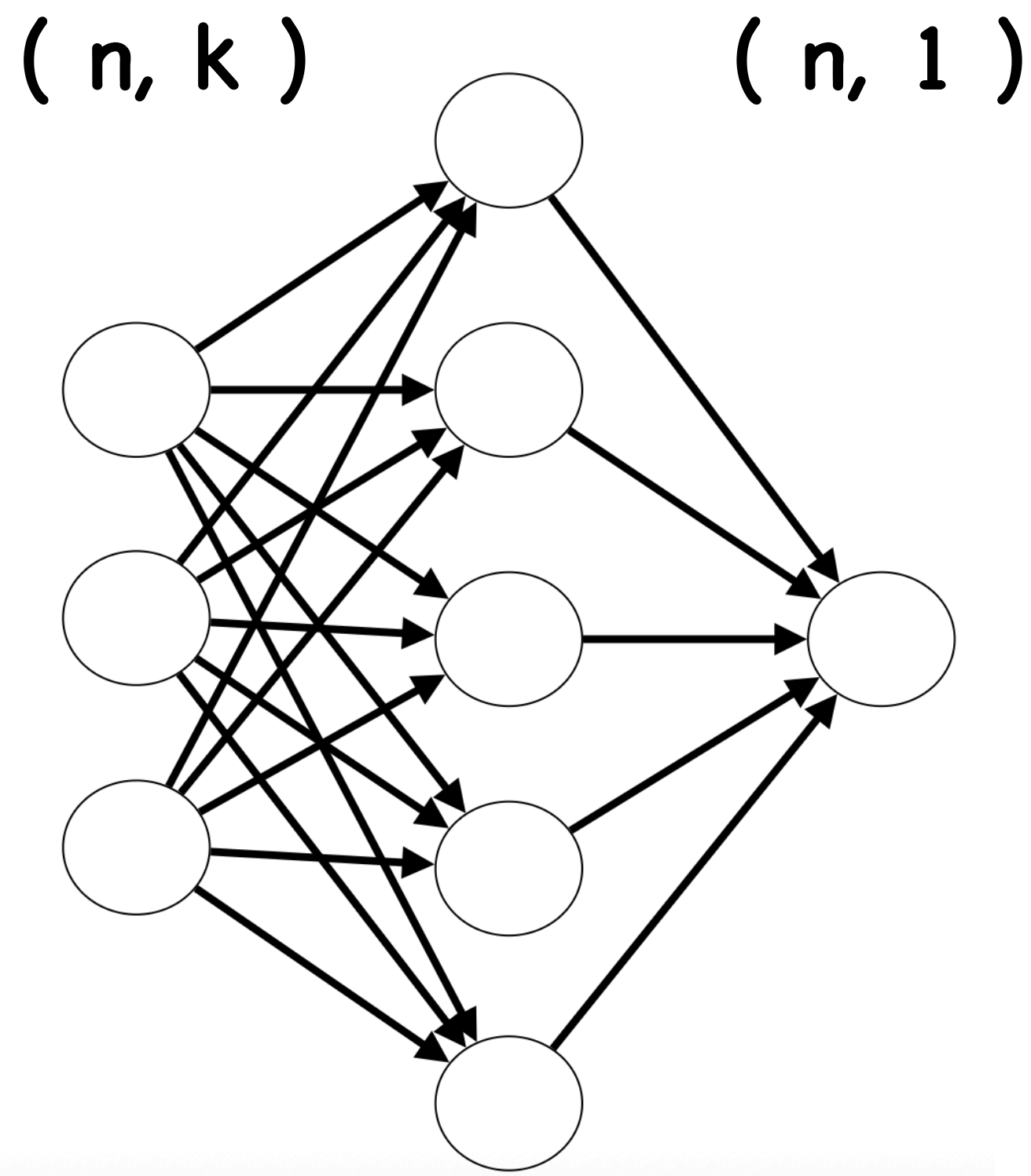


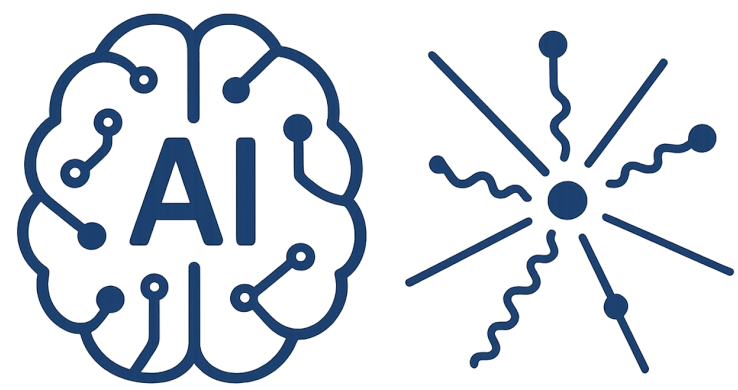
Extensions to NN



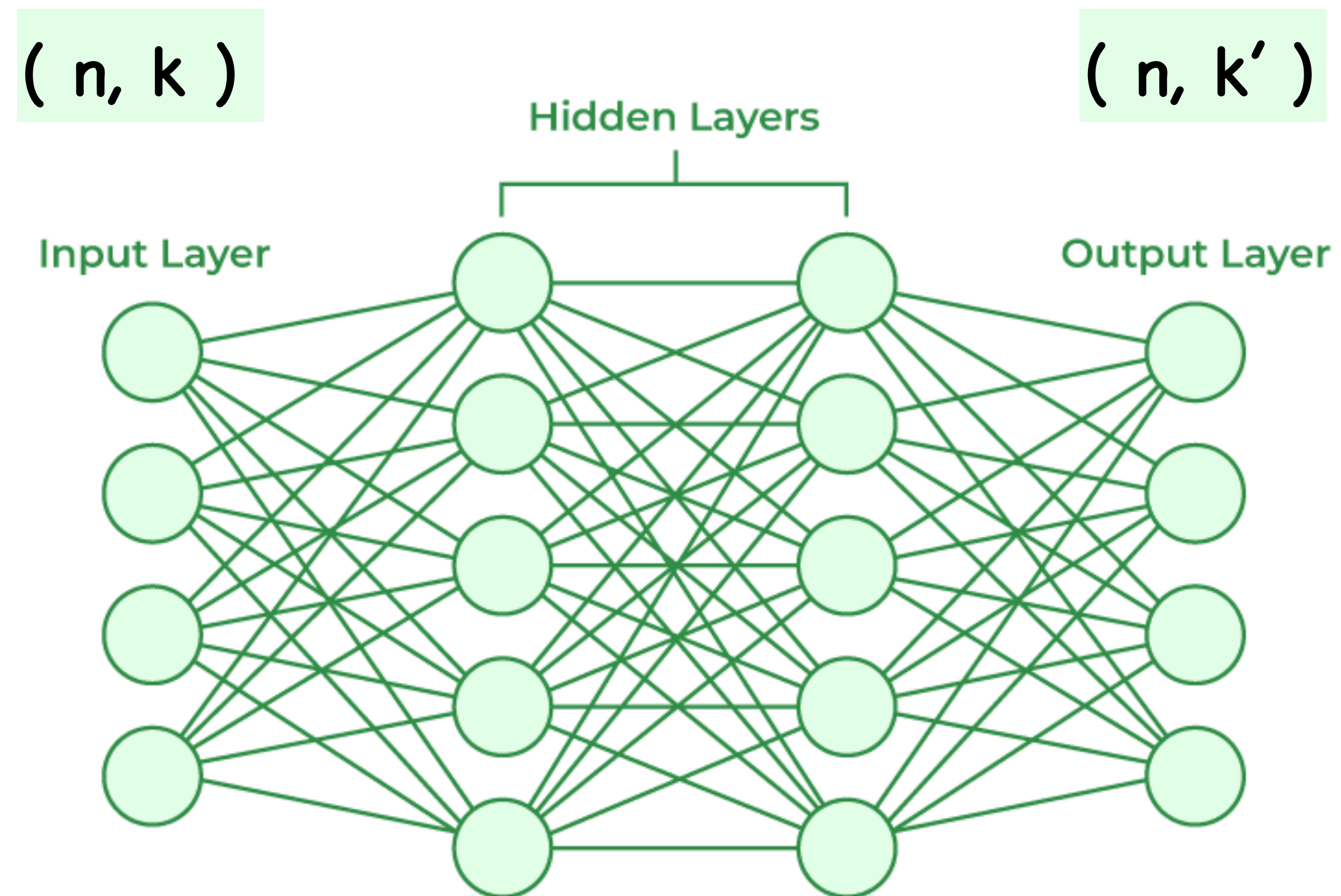


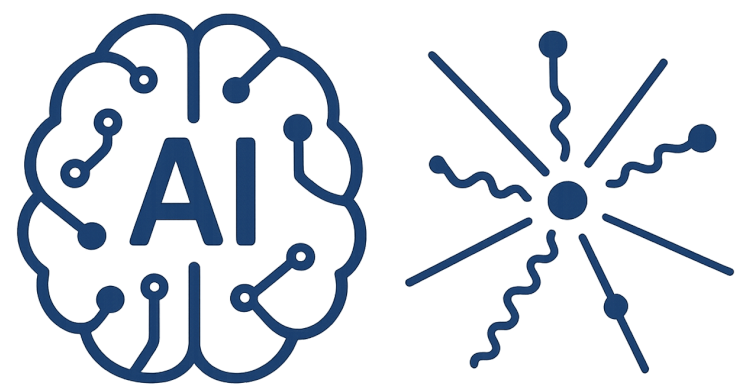
Extensions to NN





Deep NN (Deep Learning)

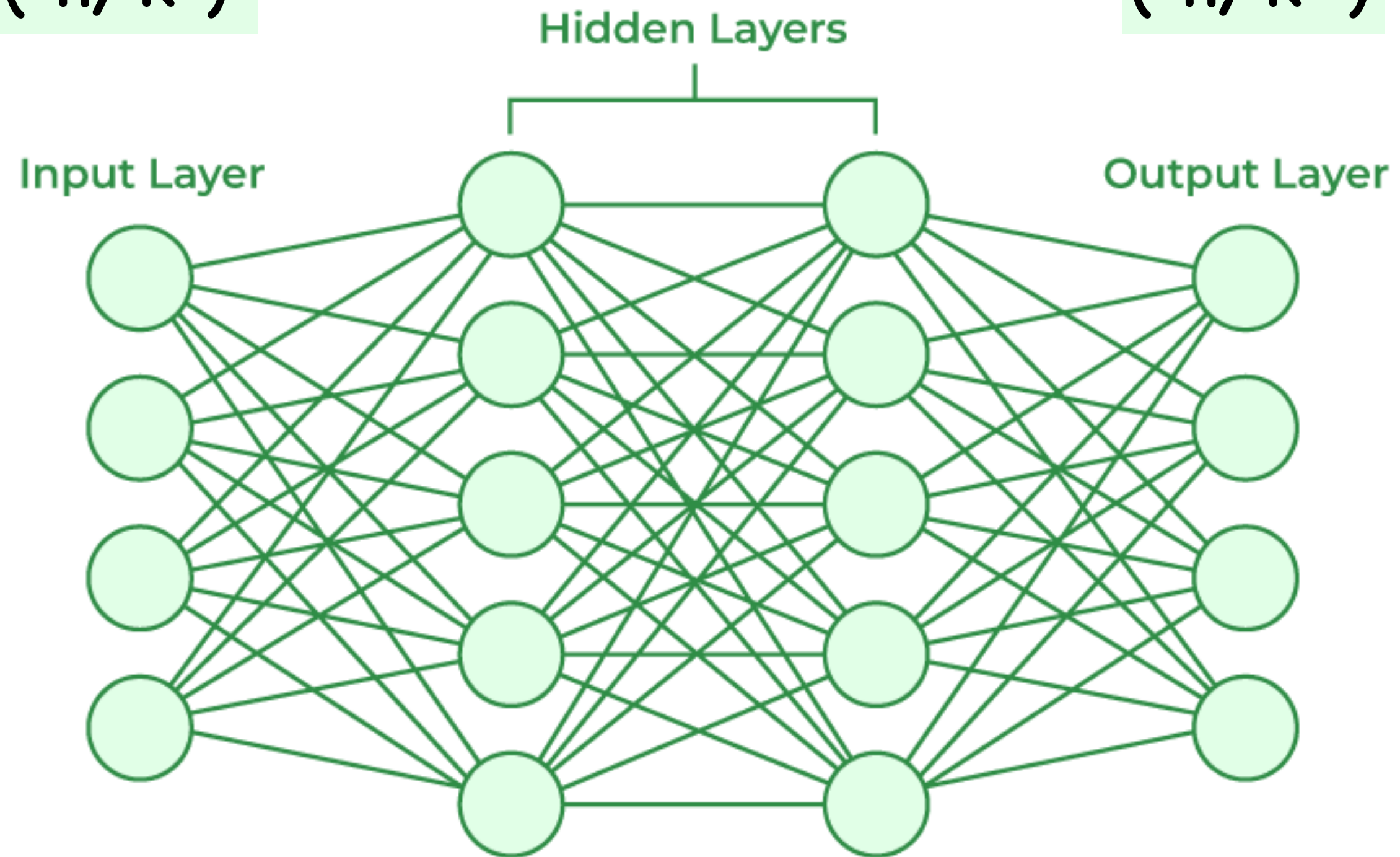




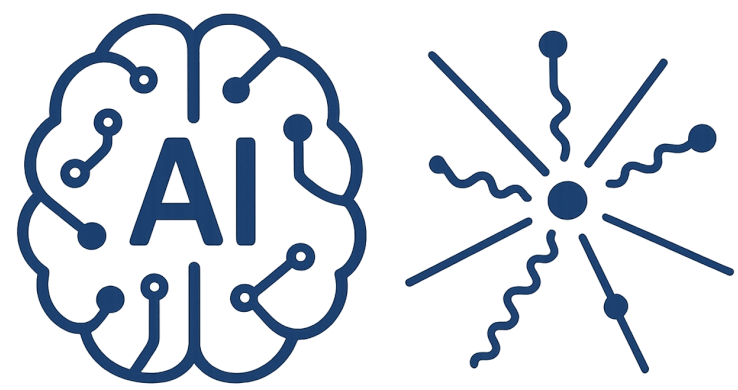
Deep NN (Deep Learning)

(n, k)

(n, k')



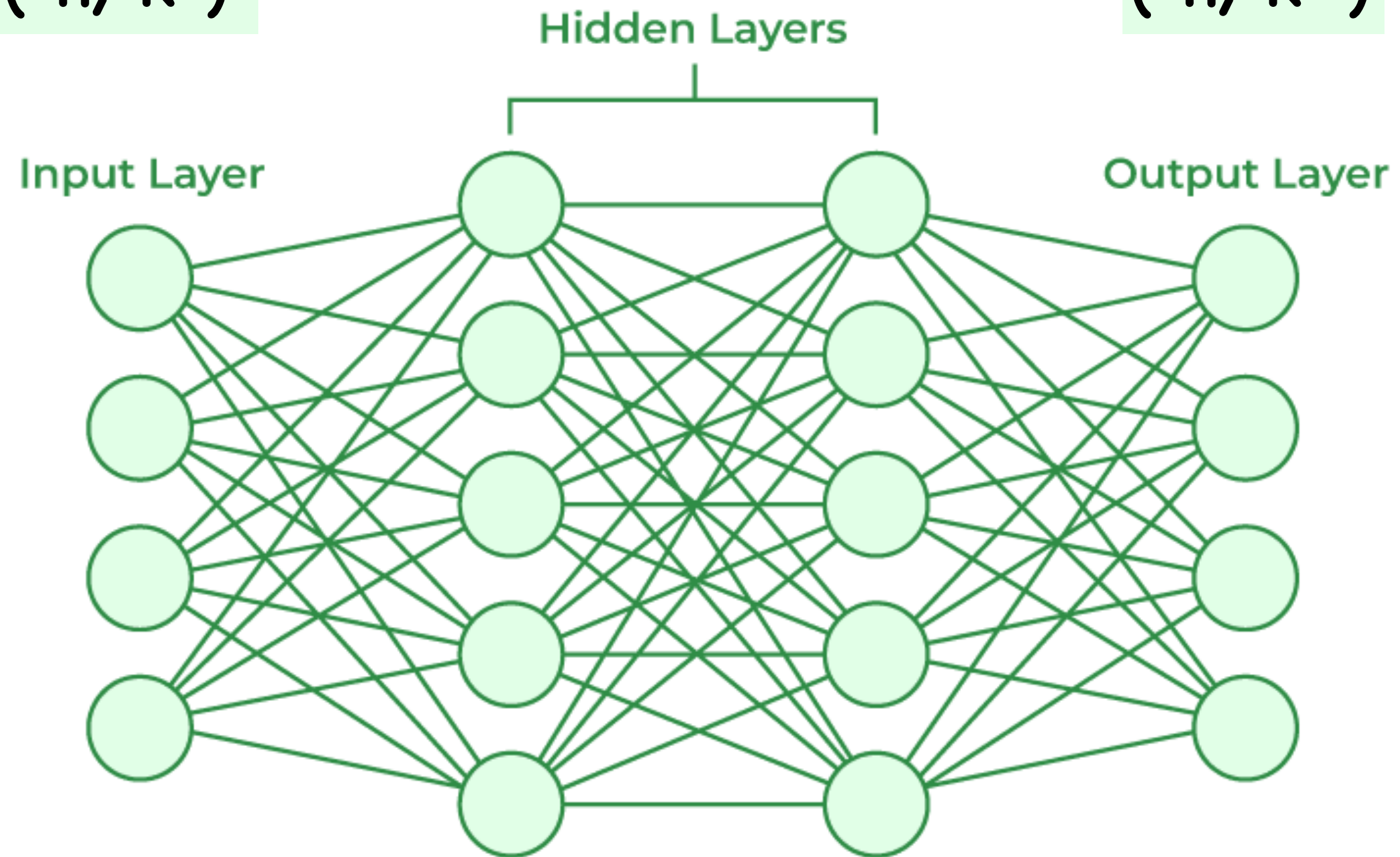
What about input with more complex data structure ?



Deep NN (Deep Learning)

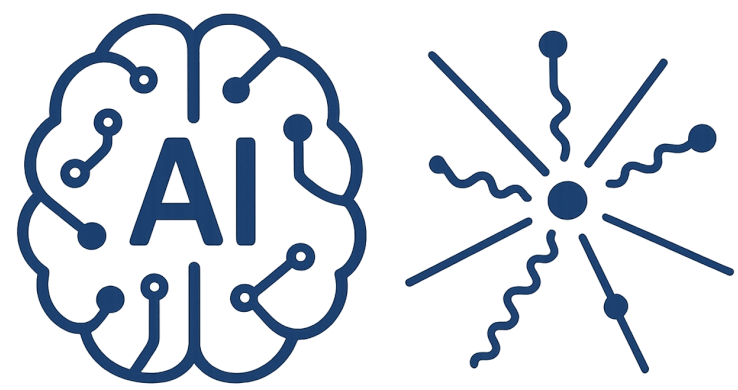
(n, k)

(n, k')

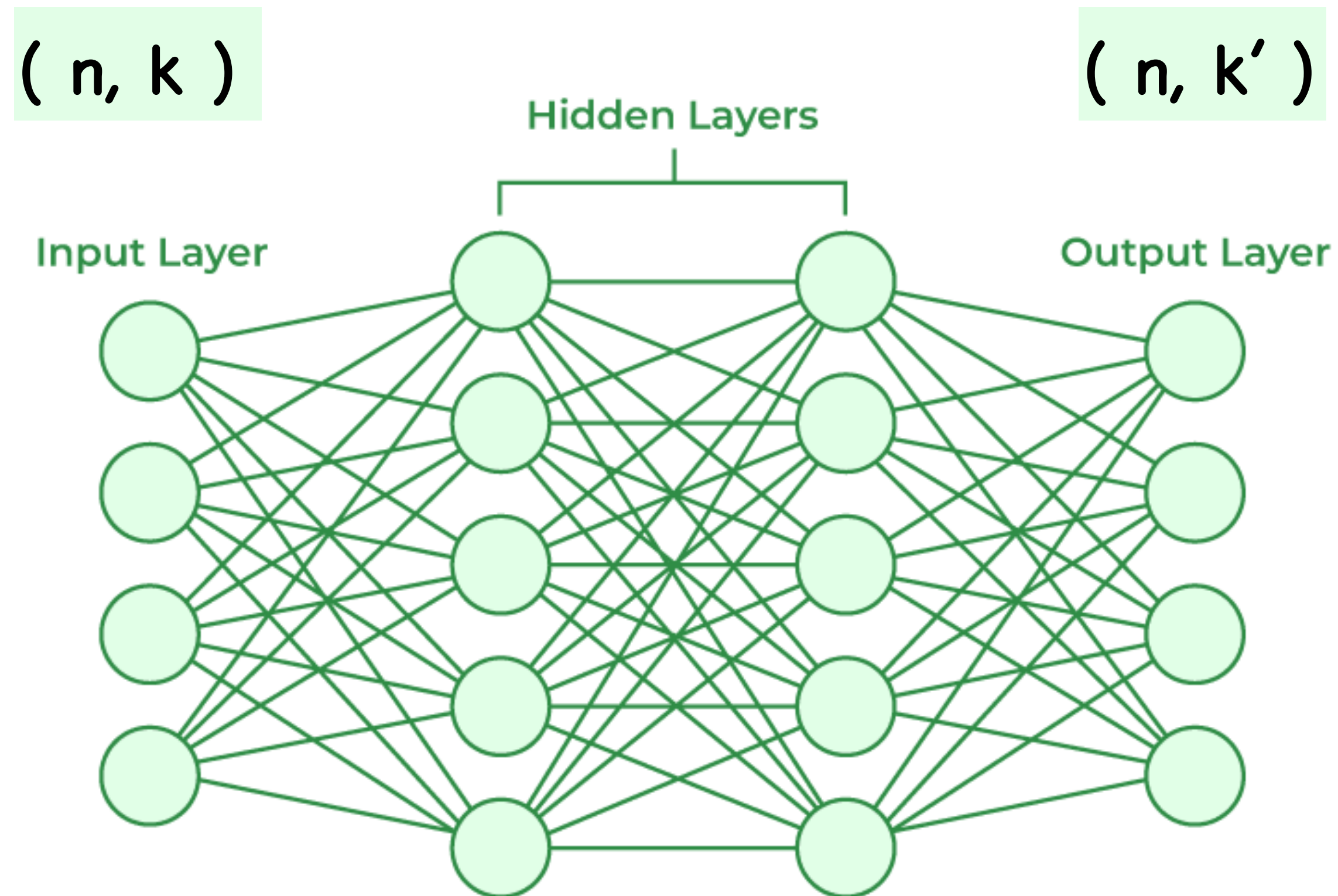


What about input with more complex data structure ?

Extract features



Deep NN (Deep Learning)

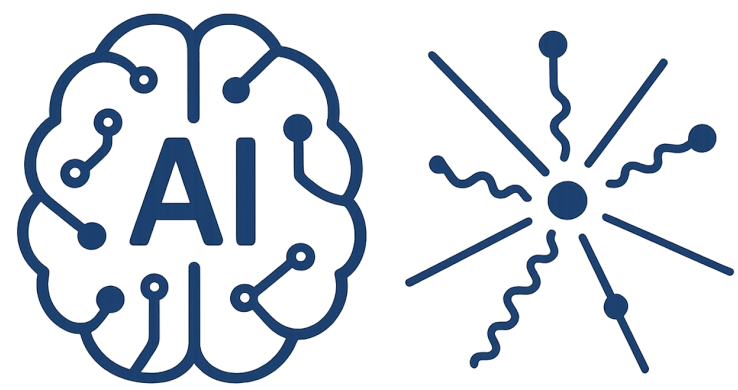


What about input with more complex data structure ?

Extract features

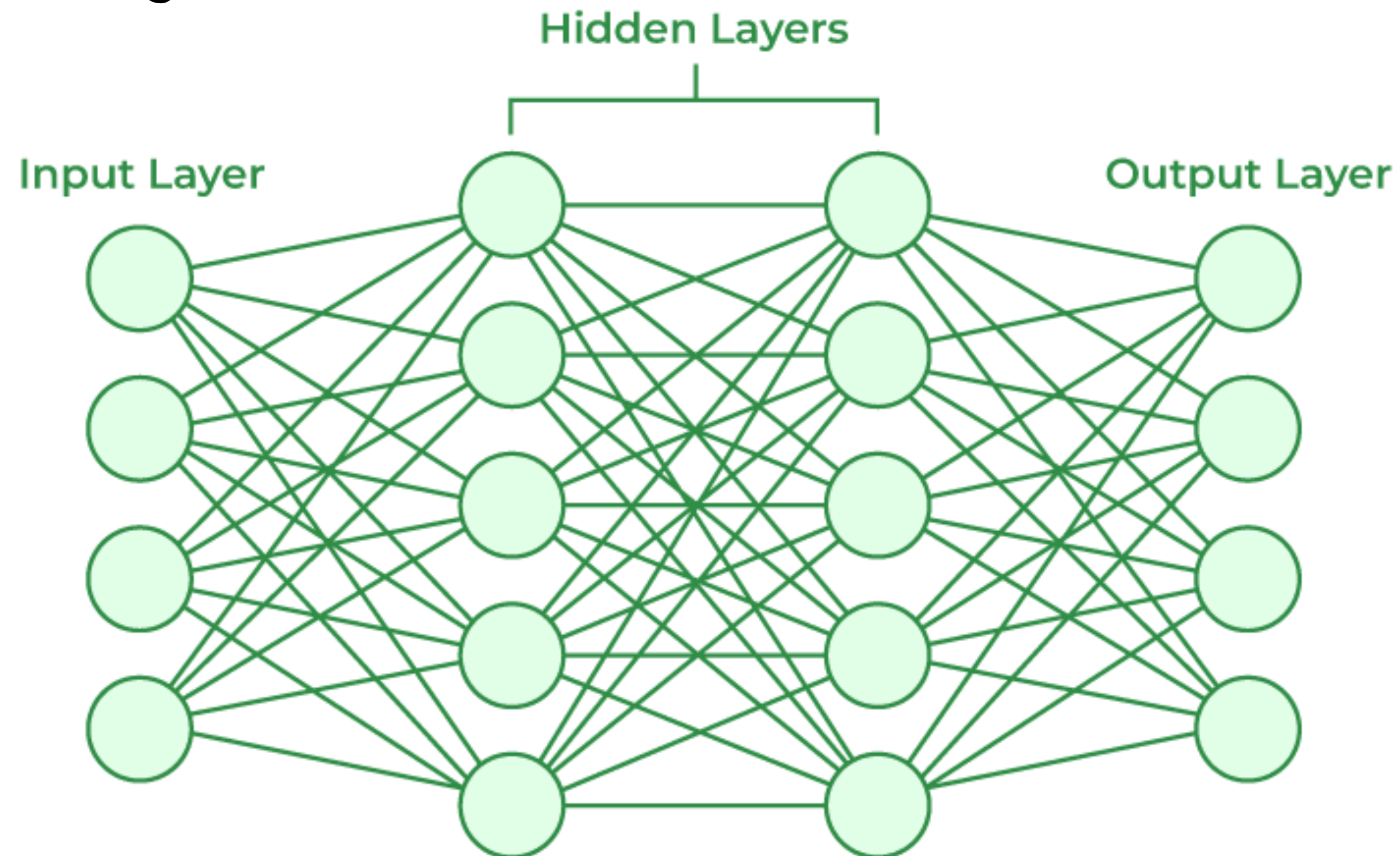
feed-forward

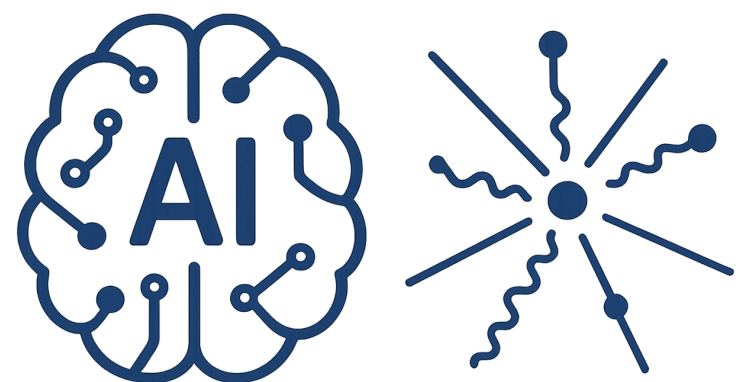
information is flowing forward through the network in the direction of the arrows.



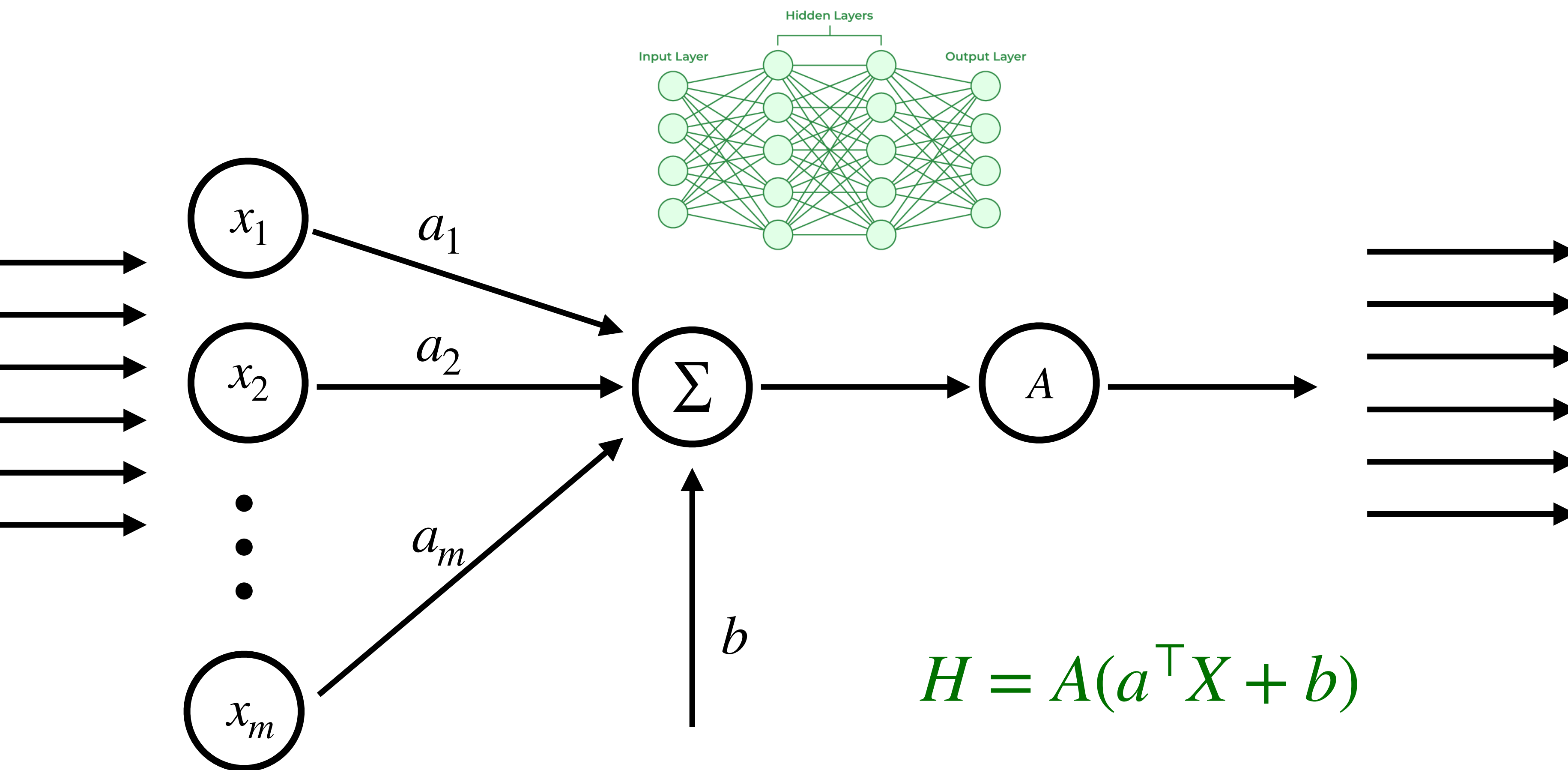
More Extensions to

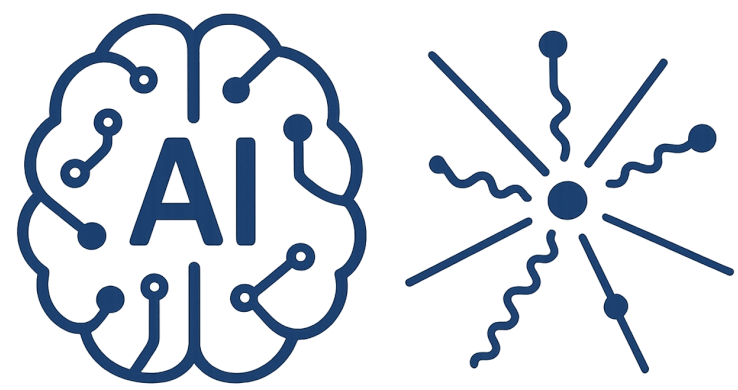
Structure of a single neuron



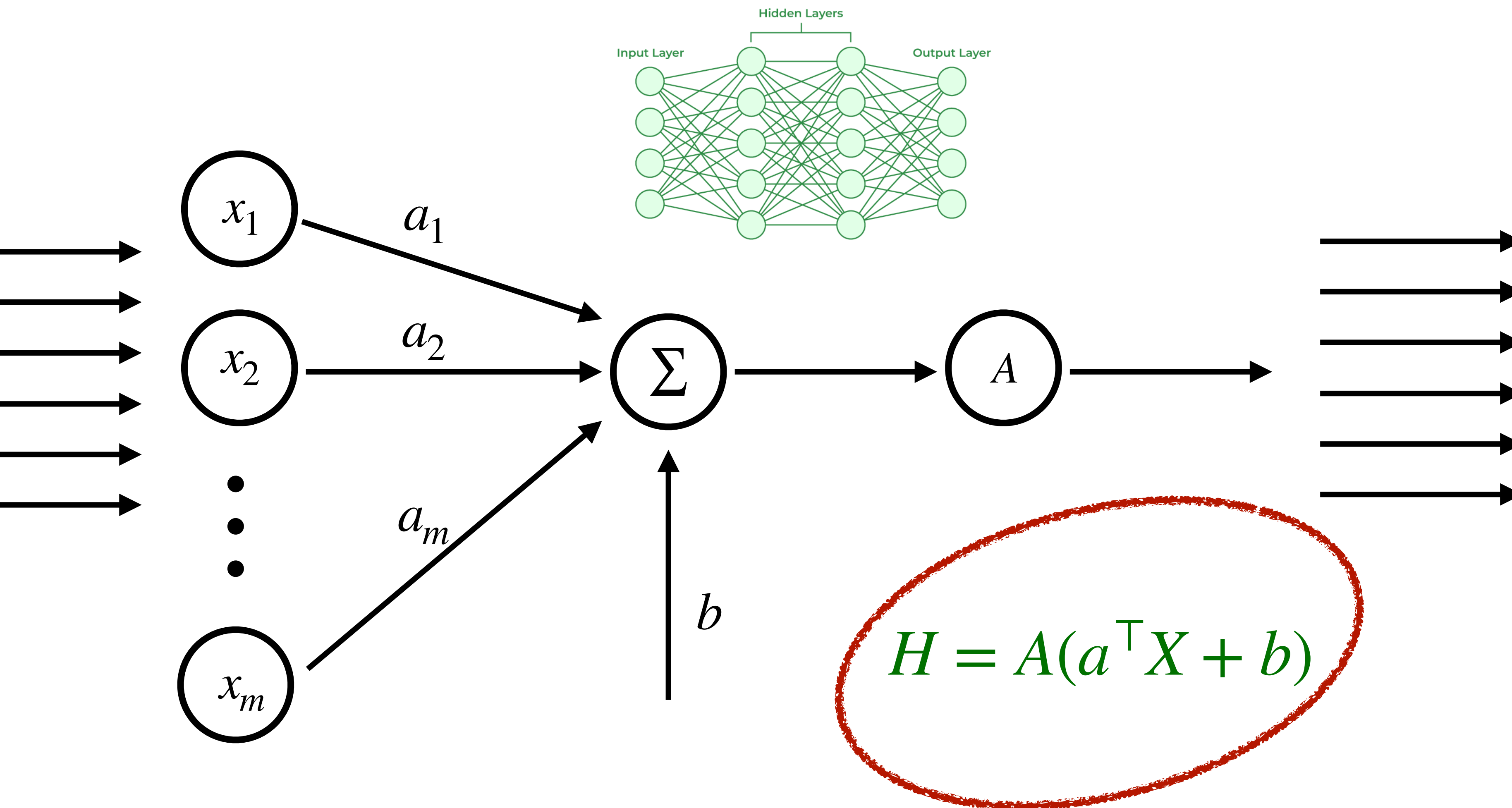


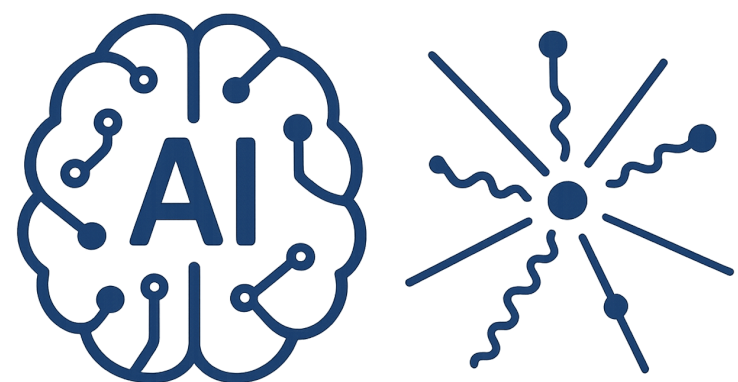
NN fundamental computational unit



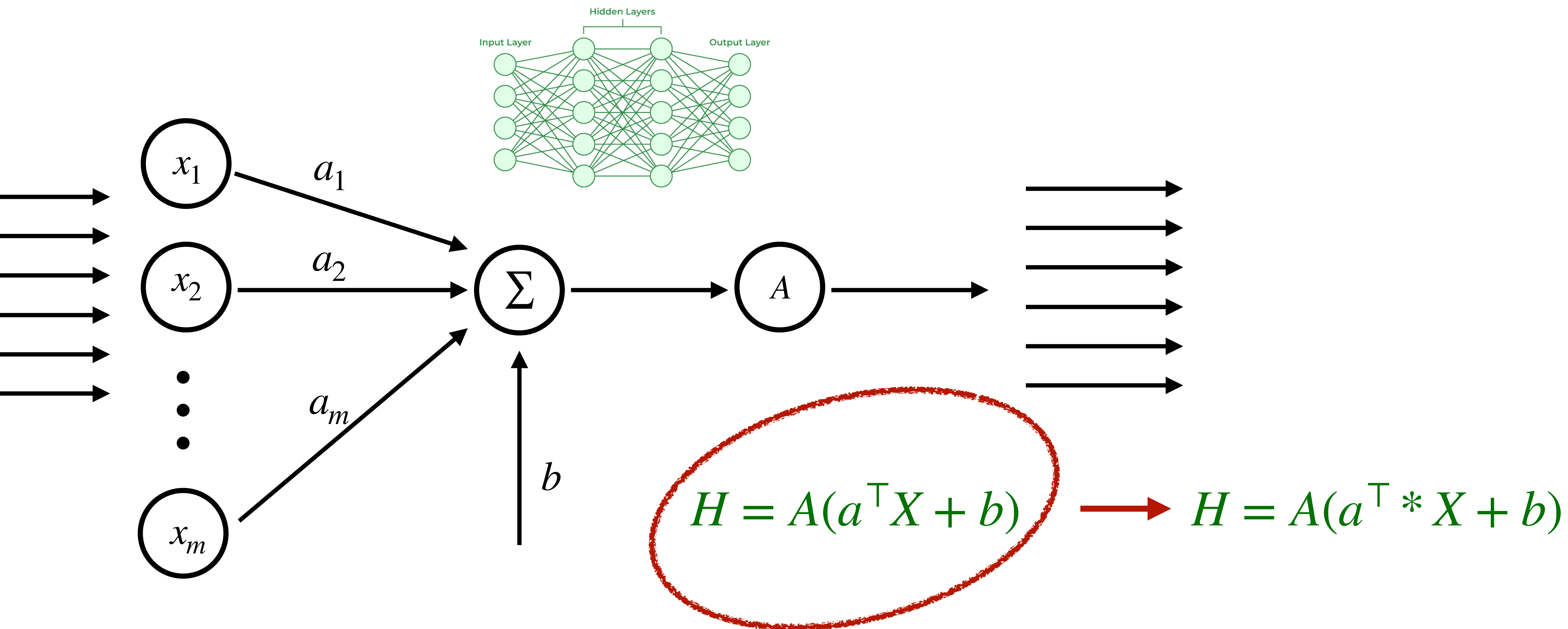


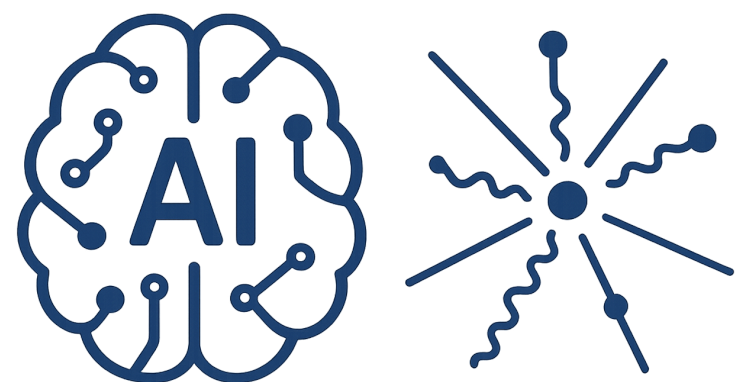
NN fundamental computational unit



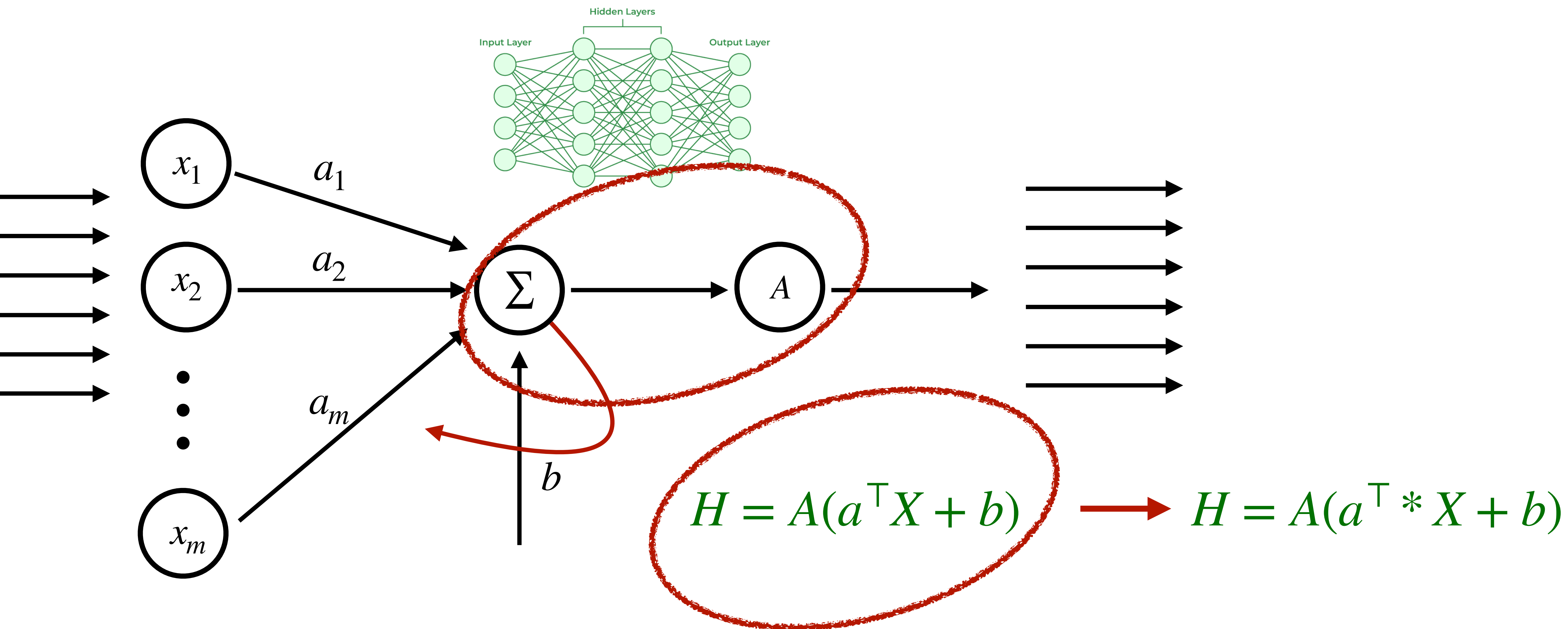


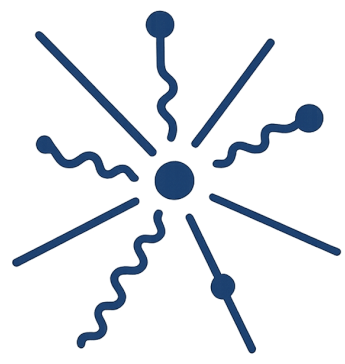
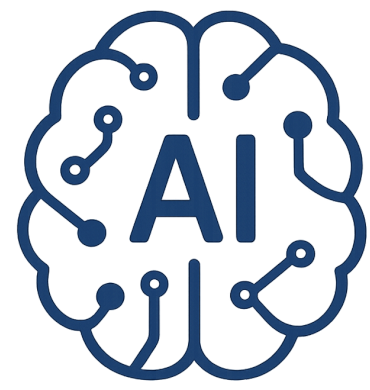
NN fundamental computational unit





NN fundamental computational unit

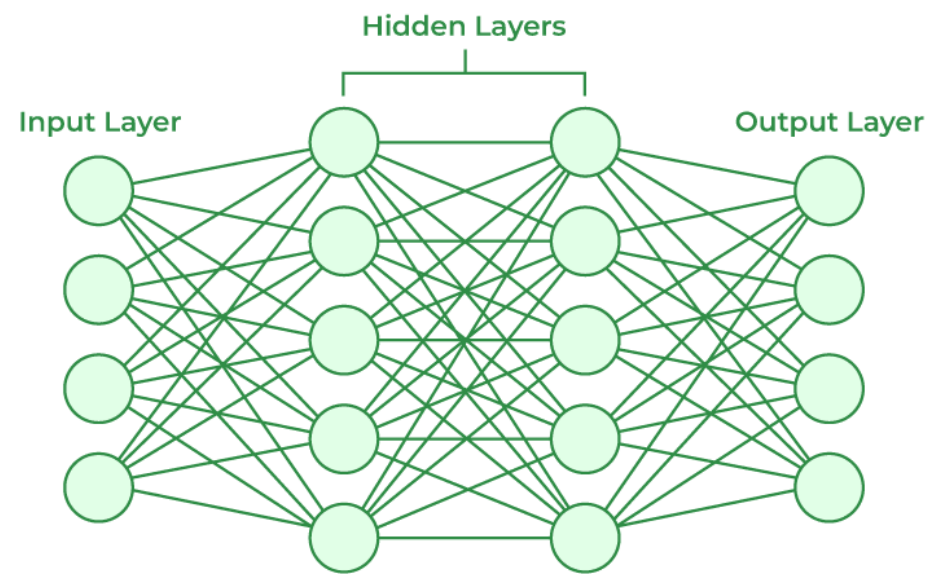




More Extensions to

Optimization

$$y = f(x; a)$$



$$L \sim \sum [\tilde{y} - f(x; a)]^2 + \lambda \|a\|^2$$

stochastic gradient descent

$$a^{i+1} = a^i - \eta \nabla_a L(a^i)$$

error backpropagation

information flows backwards through the network from the outputs towards the inputs



Optimization Algorithms



Optimization Algorithms

Motivation

Parameter estimation

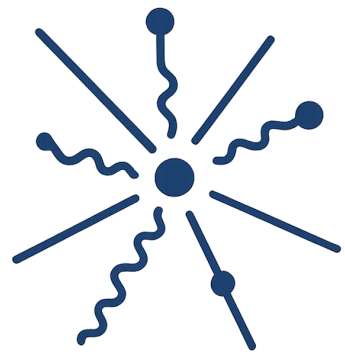
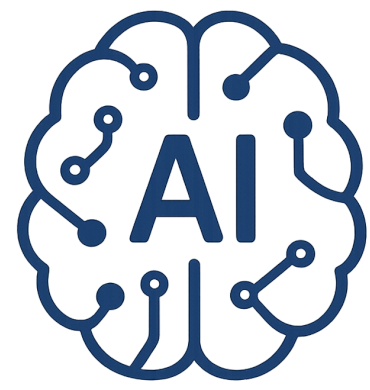
Simulation Calibration

Machine Learning Training



Optimization Algorithms

$$y = f(a)$$



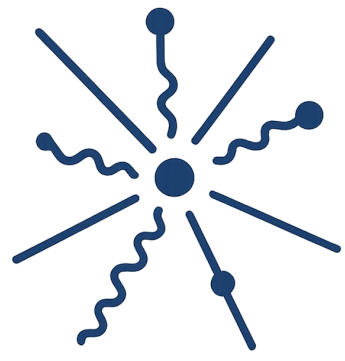
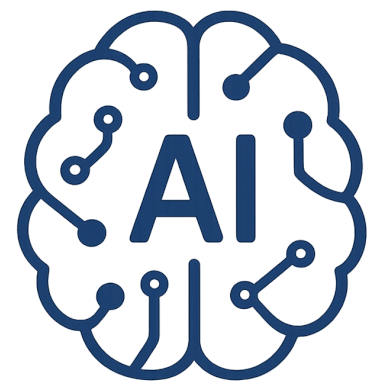
Optimization Algorithms

$$y = f(a)$$

Goal

Finding value of a that optimize $f(a)$

$$a_{\min} = \arg \min_a f(a)$$

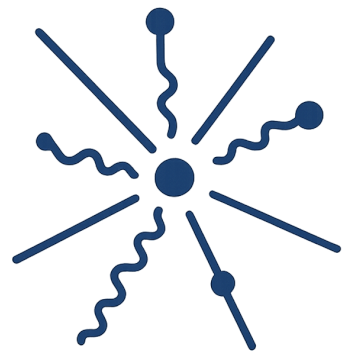
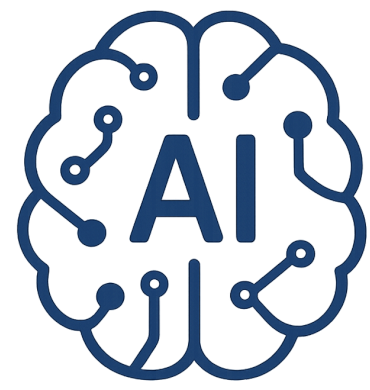


Optimization Algorithms

Approach 1: Search-Based

exploring the landscape

no need to derivative

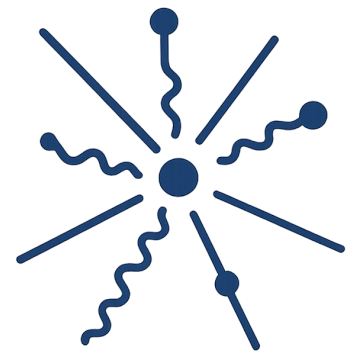
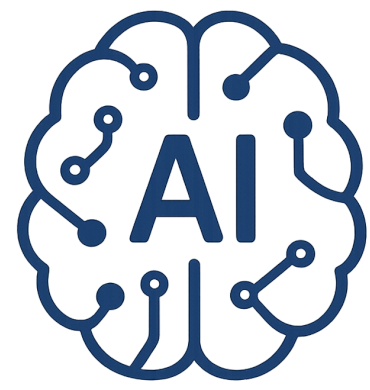


Optimization Algorithms

Approach 2: Gradient-Based

move in the opposite direction of the gradient

$$a_{new} = a - \eta f'(a_{old})$$



Optimization Algorithms

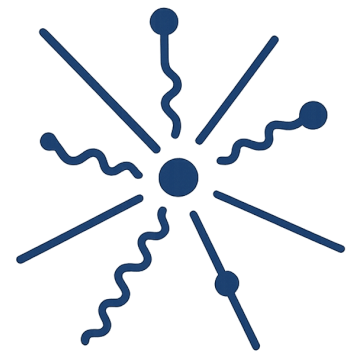
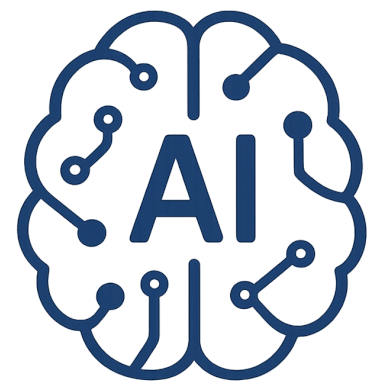
Approach 2: Gradient-Based

move in the opposite direction of the gradient

$$a_{new} = a - \eta f'(a_{old})$$

$$f(a_{new}) = f(a_{old}) + f'(a_{old})(a_{new} - a_{old}) + \frac{1}{2} f''(a_{old}) (a_{new} - a_{old})^2$$

$$f(a_{new}) - f(a_{old}) \leq \eta (f'(a_{old}))^2 \left(\frac{\eta}{2} f''(a_{old}) - 1 \right)$$

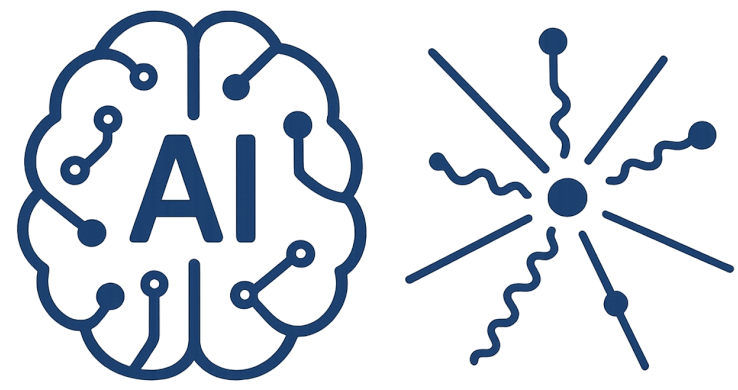


Optimization Algorithms

Approach 2: Gradient-Based

move in the opposite direction of the gradient

need to calculate derivative



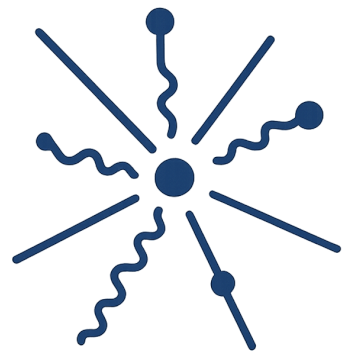
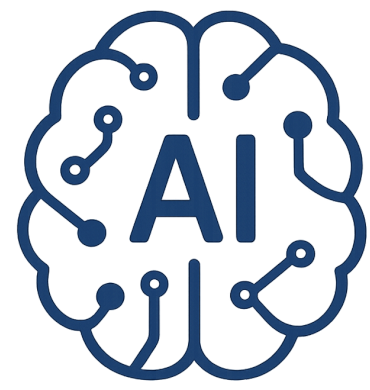
Derivative in PyTorch

```
x = torch.tensor(2.0, requires_grad=True)
```

```
y = (x ** 2 + 3 * x + 1)
```

```
y.backward()
```

```
print(x.grad)    ?
```

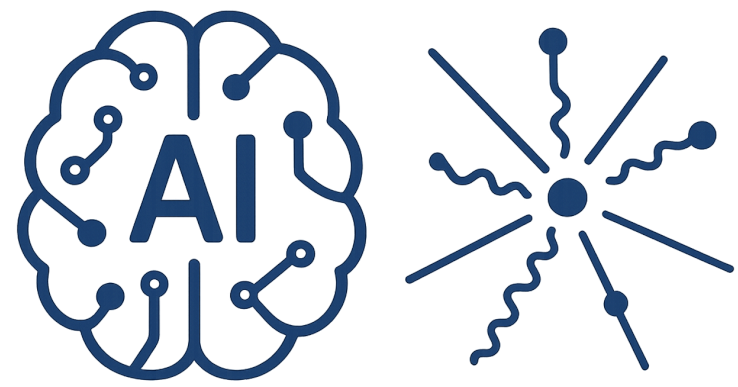
Derivative in PyTorch

```
x = torch.tensor(2.0, requires_grad=True)
```

```
y = (x ** 2 + 3 * x + 1)
```

```
y.backward()
```

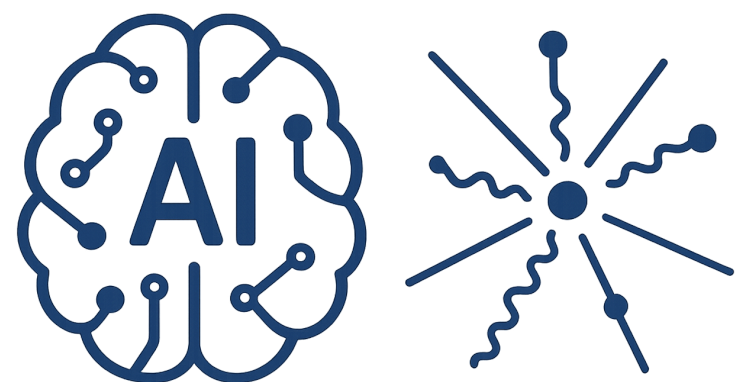
```
print(x.grad)    7
```



AI-Driven HEP 10

Derivative in PyTorch

```
def forward(x1,x2):  
    return x1 * x2 + x2  
  
def compute_gradients(x1_value, x2_value):  
  
    x1 = torch.tensor(x1_value, requires_grad=True)  
    x2 = torch.tensor(x2_value, requires_grad=True)  
  
    # Forward:  $z = x1 * x2 + x2$   
    z = forward(x1,x2)  
  
    # Backward: compute gradients  
    z.backward()  
  
    # Return results  
    return z.item(), x1.grad.item(), x2.grad.item()  
  
# Example usage:  
z, dx1, dx2 = compute_gradients(3.0, 4.0)
```



AI-Driven HEP 10

Derivative in PyTorch

```
def forward(x1,x2):  
    return x1 * x2 + x2  
  
def compute_gradients(x1_value, x2_value):  
  
    x1 = torch.tensor(x1_value, requires_grad=True)  
    x2 = torch.tensor(x2_value, requires_grad=True)  
  
    # Forward:  $z = x1 * x2 + x2$   
    z = forward(x1,x2)  
  
    # Backward: compute gradients  
    z.backward()  
  
    # Return results  
    return z.item(), x1.grad.item(), x2.grad.item()
```

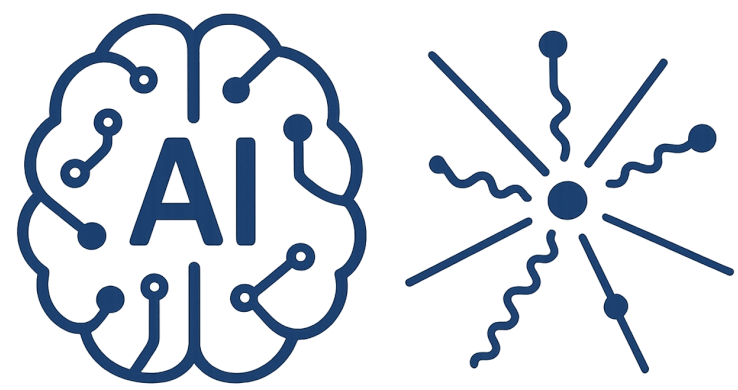
Example usage:

```
z, dx1, dx2 = compute_gradients(3.0, 4.0)
```

$z = 16.0$

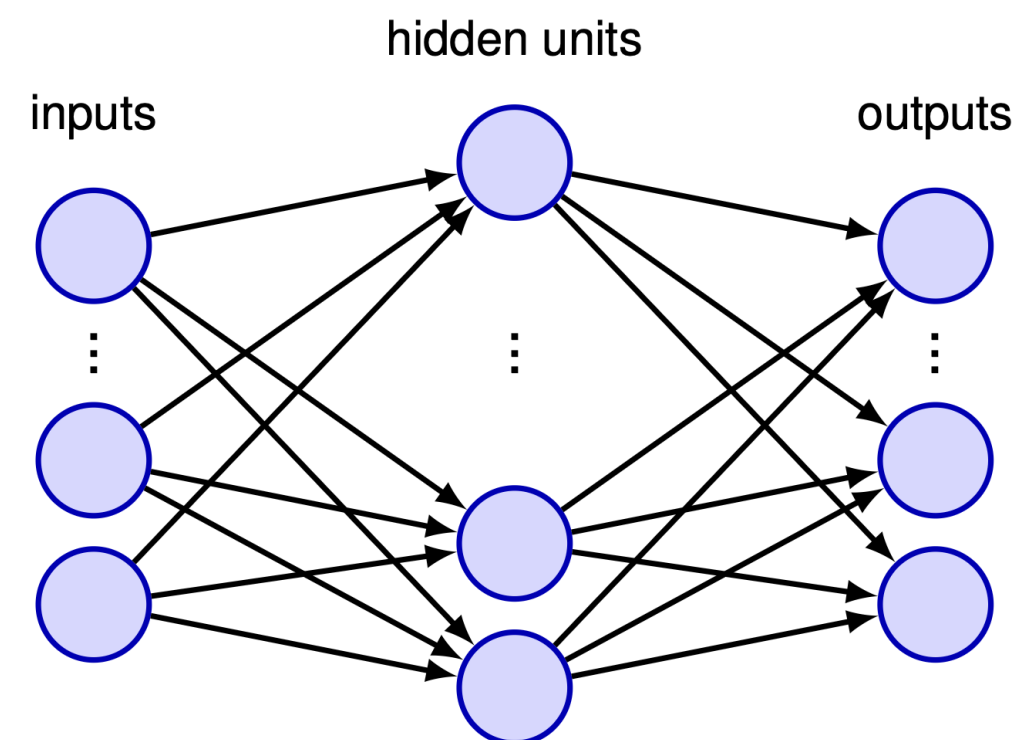
$dz/dx1 = 4.0$

$dz/dx2 = 4.0$

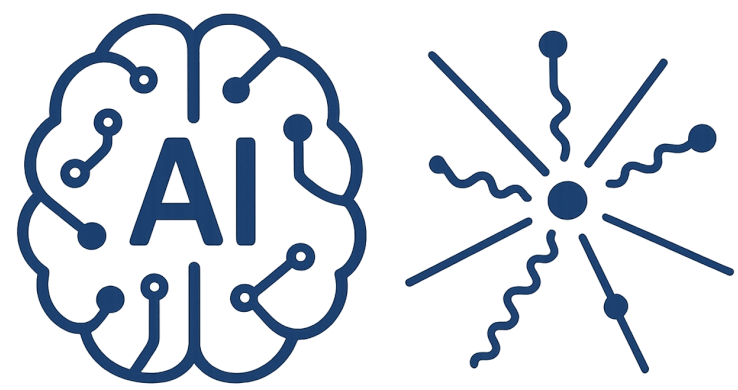


AI-Driven HEP 10

Simple NN in PyTorch

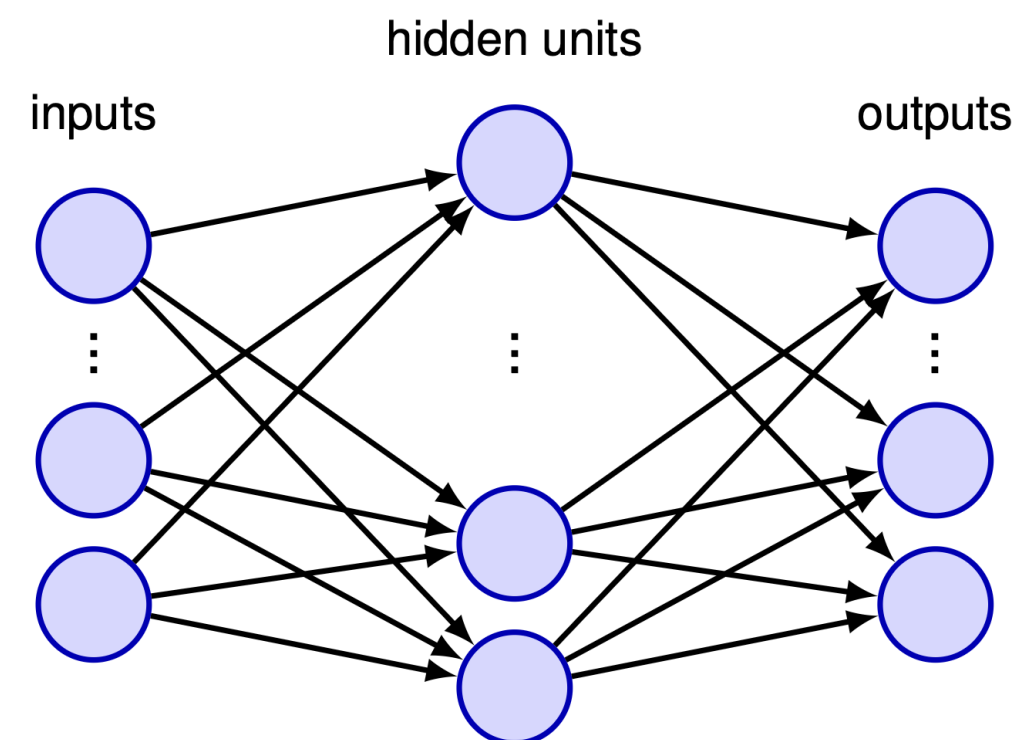


```
class SimpleNN(nn.Module):  
    def __init__(self, input_size, hidden_size, output_size):  
        super(SimpleNN, self).__init__()  
        self.hidden = nn.Linear(input_size, hidden_size)  
        self.relu = nn.ReLU()  
        self.output = nn.Linear(hidden_size, output_size)  
  
    def forward(self, x):  
        x = self.hidden(x)  
        x = self.relu(x)  
        x = self.output(x)  
        return x
```

AI-Driven HEP 10

Simple NN in PyTorch



```
model = SimpleNN(input_size=2, hidden_size=4, output_size=2)
```

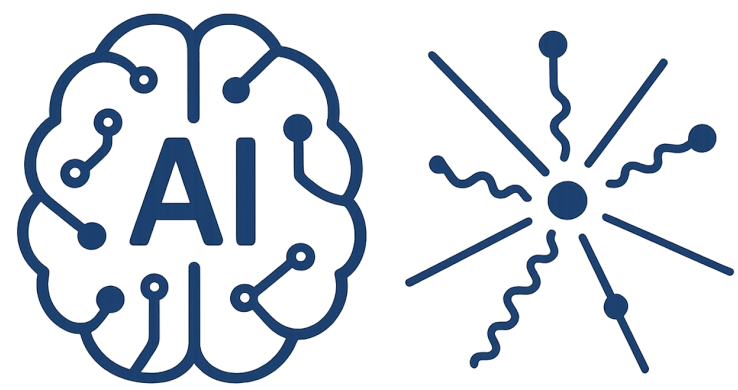
```
criterion = nn.MSELoss()
```

```
optimizer = optim.SGD(model.parameters(), lr=0.01)
```

```
x = torch.tensor([[1.0, 2.0],  
                  [2.0, 3.0],  
                  [3.0, 4.0]], requires_grad=False)
```

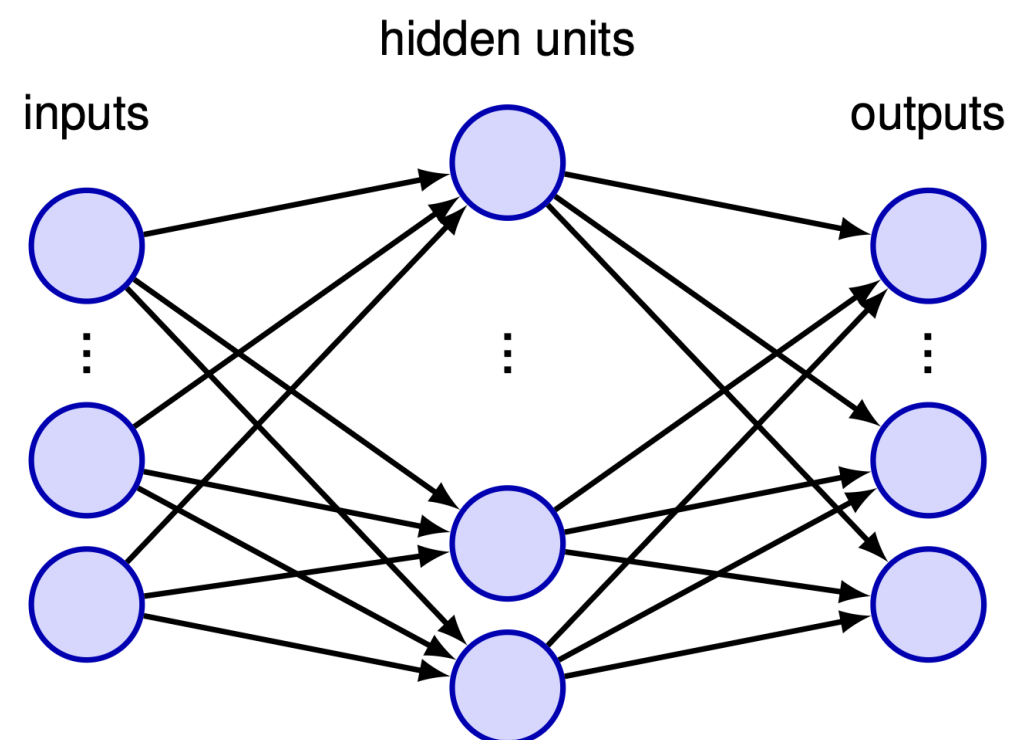
```
y = torch.tensor([[3.0, 4.0], [5.0, 6.0], [7.0, 8.0]])
```

```
for epoch in range(1000):  
    # Forward pass  
    y_pred = model(x)  
    loss = criterion(y_pred, y)  
  
    # Backward pass  
    optimizer.zero_grad()  
    loss.backward()  
    optimizer.step()
```



AI-Driven HEP 10

Simple NN in PyTorch



```
model = SimpleNN(input_size=2, hidden_size=4, output_size=2)
```

```
criterion = nn.MSELoss()
```

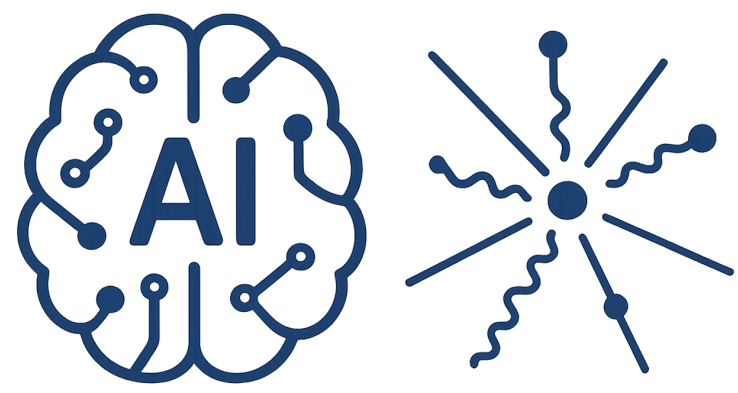
```
optimizer = optim.SGD(model.parameters(), lr=0.01)
```

```
x = torch.tensor([[1.0, 2.0],  
                  [2.0, 3.0],  
                  [3.0, 4.0]], requires_grad=False)
```

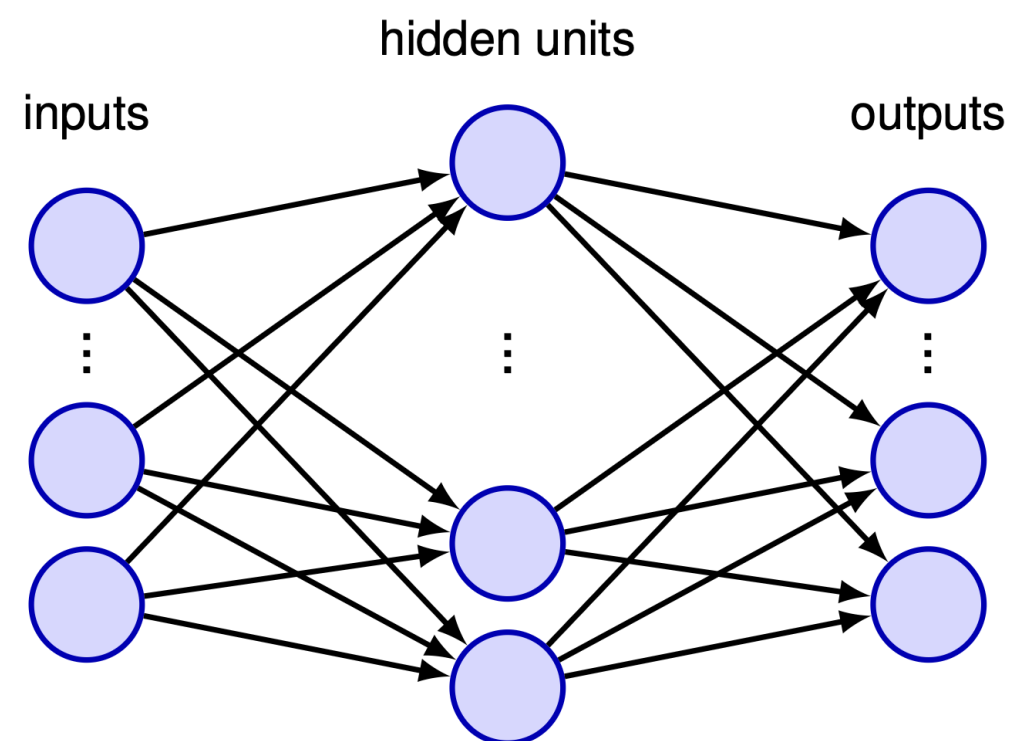
```
y = torch.tensor([[3.0, 4.0], [5.0, 6.0], [7.0, 8.0]])
```

```
for epoch in range(1000):  
    # Forward pass  
    y_pred = model(x)  
    loss = criterion(y_pred, y)  
  
    # Backward pass  
    optimizer.zero_grad()  
    loss.backward()  
    optimizer.step()
```

Predictions after training:
tensor([[3.0468, 3.9577],
 [5.0091, 5.9918],
 [6.9714, 8.0259]])



AutoGrad in PyTorch



```
model = SimpleNN(input_size=2, hidden_size=4, output_size=2)
```

```
# ---- Fix the weights and biases ----
```

```
with torch.no_grad():
```

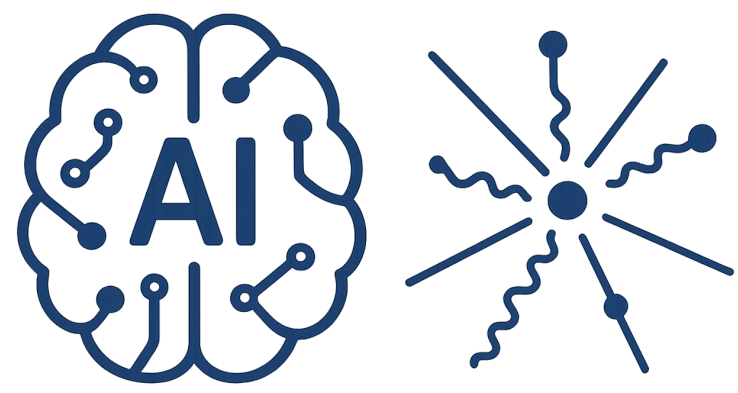
```
    model.hidden.weight.copy_(torch.tensor([
        [0.1, 0.2],
        [0.3, 0.4],
        [0.5, 0.6],
        [0.7, 0.8]])))
```

```
    model.hidden.bias.copy_(torch.tensor([0.1, 0.2, 0.3, 0.4]))
```

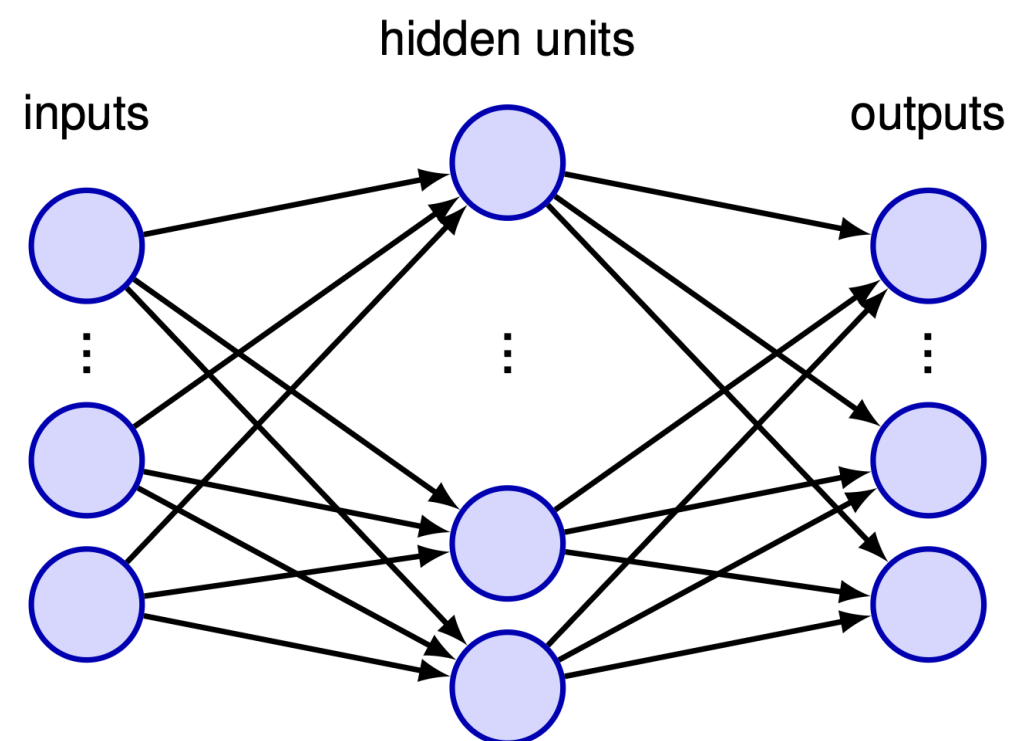
```
# Output layer
```

```
    model.output.weight.copy_(torch.tensor([
        [0.2, 0.3, 0.4, 0.5],
        [0.5, 0.4, 0.3, 0.2]])))
```

```
    model.output.bias.copy_(torch.tensor([0.1, -0.1]))
```

AutoGrad in PyTorch



```
x = torch.tensor([[1.0, 2.0],  
                  [2.0, 3.0],  
                  [3.0, 4.0]], requires_grad=True)
```

```
# ---- Forward pass ----
```

```
y = model(x)
```

```
# ---- Backward pass ----
```

```
grad_outputs = torch.tensor([[1.0, 0.0],  
                             [0.0, 1.0],  
                             [1.0, 1.0]], dtype=torch.float32)
```

```
# Compute gradients of y with respect to x
```

```
gradients = torch.autograd.grad(outputs=y, inputs=x, grad_outputs=grad_outputs)
```